

Omnicortex AI Frontend Documentation

Architecture, user flows, API integration, history management, and runtime behavior

Project folder: C:\Users\adity\OneDrive\Documents\Omnicortex-Client

1. Frontend Overview

Omnicortex AI is a React + Vite single-page application that provides three authenticated AI workflows inside one SaaS dashboard: resume generation, image generation, and medical analysis. The application uses TypeScript for typing discipline, Tailwind CSS for interface styling, Axios for API access, React Router for page navigation, and localStorage-backed state for login persistence and generated link history.

- Public routes: /login and /register
- Protected routes: /dashboard, /resume, /image, and /medical
- Primary layout: sidebar navigation, top user bar, responsive content area
- Frontend brand: Omnicortex AI with SVG logo and favicon integration

2. Technology Stack

Area	Details
Framework	React 19 with Vite 6
Language	TypeScript
Styling	Tailwind CSS
Routing	React Router DOM
HTTP client	Axios with request interceptor
Notifications	react-hot-toast
Icons	lucide-react
Preview behavior	Iframe for resume PDF preview and inline img preview for images

3. Directory and Code Organization

The frontend is intentionally separated by responsibility so API access, layout components, page screens, persistence logic, and low-level utilities remain isolated.

```
src/  
  api/           backend integration modules  
  components/    shared UI, layout, route guards, branding  
  pages/         route-level screens  
  store/         auth context and generation-history persistence  
  utils/         download helpers, constants, storage, error parsing
```

4. Authentication and Session Flow

Authentication is managed through a React context in `src/store/auth-store.tsx`. On login, the frontend posts credentials to `/auth/login`, normalizes the response into an `AuthSession` object, stores the access token plus user details in `localStorage`, and exposes the session to the rest of the app through `useAuth()`. On logout, the session state and storage entry are cleared.

Area	Details
Register API	POST <code>http://127.0.0.1:8000/auth/register</code>
Login API	POST <code>http://127.0.0.1:8000/auth/login</code>
Stored auth key	<code>omnicortex-ai-auth</code>
Token usage	Axios interceptor adds <code>Authorization: Bearer <token></code> to requests
Route guard behavior	Unauthenticated access to protected pages redirects to <code>/login</code>

5. Routing Model

Routing is declared centrally in `src/App.tsx`. `PublicRoute` prevents authenticated users from returning to `login/register`, while `ProtectedRoute` prevents unauthenticated users from entering dashboard tools.

Area	Details
Public	<code>/login</code> , <code>/register</code>
Protected	<code>/dashboard</code> , <code>/resume</code> , <code>/image</code> , <code>/medical</code>
Fallback	<code>/</code> redirects to <code>/dashboard</code> , unmatched paths render 404 screen

6. Feature Pages

Each feature page follows the same operating model: form inputs on the left, results on the right, loading states through skeleton placeholders, toast notifications for success/error, and API integration handled by dedicated modules under `src/api`.

- Resume Generator: sends raw text/plain body to `/generate-resume`, previews returned `pdf_url` in an `iframe`, stores PDF history, and allows open-in-new-tab plus download actions.
- Image Generator: sends `prompt/mode/steps` JSON to `/generate-image`, uses backend-supported mode values (`simple`, `portrait`, `widescreen_landscape`, `product_photo`), previews the image, and stores image history.
- Medical Analyzer: submits multipart form data with file and `modality_hint` to `/analyze-medical`, then renders a structured result card with preview image, findings, conditions, recommendation, and disclaimer.

7. URL History and Persistence

The frontend preserves generated links and page state in `localStorage` through `src/store/generation-history.ts`. This was added so navigation between Resume and Image pages does not discard previously generated outputs. Each page restores its last-known form values and most recent result on load, and it also maintains a clickable history list.

Area	Details
Resume state key	<code>omnicortex-ai-resume-page</code>

Image state key	omnicortex-ai-image-page
History size cap	8 items per tool
Resume history fields	id, createdAt, input, pdfUrl, result
Image history fields	id, createdAt, prompt, mode, steps, imageUrl

Resume and image pages call `getResumePageState()/getImagePageState()` during initialization, then use `useEffect()` to keep the current page state synchronized back into `localStorage` whenever inputs, result URLs, or history arrays change. History appenders also deduplicate by URL before trimming to the maximum size.

8. Download Management

The current download implementation uses a same-origin proxy path instead of direct cross-origin file saving. This is defined in `src/utls/download.ts` and supported by a Vite middleware in `vite.config.ts`. The frontend rewrites download requests to `/__download?url=<remote-url>&filename=<name>`, and the Vite server fetches the remote file, applies a safe filename, and returns it with `Content-Disposition` set for browser download.

- Reason for proxy: remote object storage URLs may preview correctly but still fail direct browser download due to cross-origin restrictions or missing `Content-Disposition`.
- Client helper: `buildDownloadProxyUrl()` generates the same-origin route.
- Client trigger: `downloadFromUrl()` creates an anchor against the proxy URL so the browser treats it as a first-party file download.
- Filename handling: `getFilenameFromUrl()` extracts a readable default filename from the returned URL path.

9. User Interface and Styling

The UI is designed as a restrained SaaS dashboard rather than a marketing landing page. Tailwind utility classes define spacing, borders, typography, and responsive behavior. Shared components include `Button`, `Card`, `Input`, `Select`, `Textarea`, `Skeleton`, `Badge`, `PageHeader`, `EmptyState`, `AuthShell`, and `AppShell`. This keeps the styling consistent across all workflows and reduces duplicated page-specific markup.

10. Error Handling and Feedback

All API pages use toast notifications for visible status feedback. The helper in `src/utls/error.ts` normalizes backend and Axios errors into short user-facing strings. Loading conditions are surfaced through button spinners and skeleton placeholders so the user can distinguish waiting states from broken states.

11. How to Run and Build

Area	Details
Development	<code>npm run dev</code> or <code>corepack pnpm dev</code>
Type check	<code>tsc -b</code>
Production build	<code>vite build</code>
Server host	<code>127.0.0.1:5173</code>
Backend base URL	<code>http://127.0.0.1:8000</code>

12. Practical Maintenance Notes

- If backend auth response changes, login normalization logic in `src/api/auth.ts` is the adjustment point.

- If new tools are added, add a page, API module, sidebar item, and protected route entry.
- If more persisted tools are introduced, follow the same localStorage pattern used in `src/store/generation-history.ts`.
- If download behavior changes again, check both `src/utls/download.ts` and the `/__download` middleware in `vite.config.ts`.

Prepared for Omnicortex AI frontend handoff and maintenance.